

Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων

Μάθημα: Ανάπτυξη Εφαρμογών Παγκόσμιου Ιστού

Ομάδα

Μπόνης Ματθαίος – 1115202300136
Φραντζιάς Αντρέας – 1115202300234

Σεπτέμβριος 2026

Περιεχόμενα

1	Εισαγωγή	2
2	Αρχιτεκτονική Συστήματος	2
2.1	Frontend	2
2.2	Backend	2
2.3	Βάση Δεδομένων	2
2.4	Ασφάλεια και HTTPS	2
3	Σχεδίαση Βάσης Δεδομένων	3
4	Ρόλοι και Αυθεντικοποίηση	3
5	Λειτουργίες Εφαρμογής	4
5.1	Καλωσόρισμα, Σύνδεση και Εγγραφή	4
5.2	Διαχείριση Χρηστών	4
5.3	Διαχείριση Εκδηλώσεων	4
5.4	Αναζήτηση και Προβολή Εκδηλώσεων	4
5.5	Κρατήσεις	4
5.6	Μηνύματα	4
5.7	Εξαγωγή XML και JSON	5
6	Αλγόριθμος Συστάσεων	5
7	Εγκατάσταση και Εκτέλεση	5
8	Έλεγχος και Δοκιμές	6
9	Παραδοχές και Σχεδιαστικές Αποφάσεις	6
10	Δυσκολίες	7
11	Συμπέρασμα	7

1 Εισαγωγή

Στην εργασία υλοποιήθηκε μία πλήρης web εφαρμογή για τη διαχείριση εκδηλώσεων και την ηλεκτρονική κράτηση εισιτηρίων ή θέσεων. Η εφαρμογή ακολουθεί την αρχιτεκτονική browser/server, όπου το frontend εκτελείται στον φυλλομετρητή και επικοινωνεί με το backend μέσω REST API.

Ο βασικός στόχος ήταν να καλυφθούν οι ρόλοι και οι λειτουργίες που ζητούνται στην εκφώνηση: επισκέπτης, συμμετέχων, διοργανωτής και διαχειριστής. Παράλληλα, υλοποιήθηκαν μηχανισμοί αυθεντικοποίησης, εξουσιοδότησης, διαχείρισης χωρητικότητας, ανταλλαγής μηνυμάτων, εξαγωγής δεδομένων και συστάσεων εκδηλώσεων.

Η υλοποίηση χωρίζεται σε δύο βασικά μέρη:

- **Frontend:** εφαρμογή Angular για τη διεπαφή χρήστη.
- **Backend:** NestJS REST API με βάση δεδομένων PostgreSQL μέσω Prisma ORM.

2 Αρχιτεκτονική Συστήματος

Η εφαρμογή σχεδιάστηκε ως τυπική εφαρμογή παγκόσμιου ιστού με διαχωρισμό frontend και backend. Τα δύο μέρη επικοινωνούν αποκλειστικά μέσω HTTPS αιτημάτων προς το REST API.

2.1 Frontend

Το frontend υλοποιήθηκε με Angular και TypeScript. Περιλαμβάνει σελίδες για καλωσόρισμα, σύνδεση, εγγραφή, αναζήτηση εκδηλώσεων, προβολή λεπτομερειών, κράτηση, διαχείριση εκδηλώσεων, μηνύματα και λειτουργίες διαχειριστή. Για τους χάρτες χρησιμοποιήθηκε η βιβλιοθήκη Leaflet με OpenStreetMap, τόσο στις λεπτομέρειες εκδήλωσης όσο και στην επιλογή τοποθεσίας κατά την εγγραφή και τη δημιουργία εκδήλωσης.

Η επικοινωνία με το backend γίνεται μέσα από ξεχωριστά services ανά περιοχή του API (π.χ. EventsApiService, BookingsApiService, MessagingApiService), ώστε κάθε URL να ορίζεται σε ένα μόνο σημείο. Ένας AuthInterceptor προσθέτει αυτόματα το JWT token στην κεφαλίδα Authorization κάθε αιτήματος και, αν το backend απαντήσει με 401, αποσυνδέει τον χρήστη και τον μεταφέρει στη σελίδα σύνδεσης. Οι σελίδες προστατεύονται επιπλέον με τα route guards AuthGuard και RoleGuard.

2.2 Backend

Το backend υλοποιήθηκε με NestJS και παρέχει REST API. Είναι υπεύθυνο για την επιχειρησιακή λογική, την πρόσβαση στη βάση δεδομένων, τους ελέγχους ρόλων, την έκδοση JWT tokens, την εξαγωγή XML/JSON και τον αλγόριθμο συστάσεων. Τα δεδομένα εισόδου ελέγχονται με DTOs και class-validator, ώστε λανθασμένα αιτήματα να απορρίπτονται με 400.

2.3 Βάση Δεδομένων

Η βάση δεδομένων είναι PostgreSQL. Η πρόσβαση γίνεται μέσω Prisma ORM, το οποίο αντιστοιχίζει το σχεσιακό μοντέλο σε TypeScript αντικείμενα και διευκολύνει τις συναλλαγές (transactions), τους περιορισμούς και τις συσχετίσεις.

2.4 Ασφάλεια και HTTPS

Όλες οι αλληλεπιδράσεις εκτελούνται μέσω HTTPS. Το script εκκίνησης δημιουργεί self-signed certificates για τοπική χρήση, ώστε να καλυφθεί η απαίτηση για SSL/TLS κατά την ανάπτυξη και την επίδειξη της εφαρμογής. Οι κωδικοί αποθηκεύονται ως hash με bcrypt και δεν επιστρέφονται ποτέ από το API.

3 Σχεδίαση Βάσης Δεδομένων

Το σχεσιακό μοντέλο σχεδιάστηκε με βάση το DTD της εκφώνησης και τις επιπλέον λειτουργίες της εφαρμογής. Τα βασικά μοντέλα είναι τα ακόλουθα:

Οντότητα	Περιγραφή
User	Αποθηκεύει στοιχεία χρήστη, ρόλους, κατάσταση έγκρισης, στοιχεία επικοινωνίας, διεύθυνση, ΑΦΜ και γεωγραφική θέση.
Event	Περιγράφει την εκδήλωση με τίτλο, τύπο, χώρο, διεύθυνση, πόλη, χώρα, συντεταγμένες, ημερομηνίες, χωρητικότητα, κατάσταση και περιγραφή.
Category	Επιτρέπει σε κάθε εκδήλωση να ανήκει σε μία ή περισσότερες κατηγορίες.
TicketType	Περιγράφει τύπους εισιτηρίων με όνομα, τιμή, ποσότητα και διαθέσιμο υπόλοιπο.
Booking	Αποθηκεύει τις κρατήσεις, τον συμμετέχοντα, το είδος εισιτηρίου, τον αριθμό εισιτηρίων, το κόστος και την κατάσταση κράτησης.
Message	Υποστηρίζει τα εισερχόμενα, απεσταλμένα, μη αναγνωσμένα μηνύματα και τη διαγραφή ανά χρήστη.
EventView	Καταγράφει προβολές εκδηλώσεων από χρήστες και χρησιμοποιείται στον αλγόριθμο συστάσεων.
Photo	Αποθηκεύει προαιρετικές φωτογραφίες ή URLs φωτογραφιών για κάθε εκδήλωση.

Οι καταστάσεις ορίζονται ως enums:

- **UserStatus:** PENDING, APPROVED, REJECTED
- **EventStatus:** DRAFT, PUBLISHED, COMPLETED, CANCELLED
- **BookingStatus:** PENDING, CONFIRMED, CANCELLED

Η χωρητικότητα των εκδηλώσεων ελέγχεται στο backend. Δεν επιτρέπεται το άθροισμα των εισιτηρίων να υπερβαίνει τη συνολική χωρητικότητα και δεν επιτρέπεται κράτηση όταν δεν υπάρχουν διαθέσιμα εισιτήρια για τον ζητούμενο τύπο.

4 Ρόλοι και Αυθεντικοποίηση

Η εφαρμογή υποστηρίζει τους ρόλους που ζητούνται στην εκφώνηση:

- **Επισκέπτης:** μπορεί να βλέπει και να αναζητά δημοσιευμένες εκδηλώσεις, χωρίς να κάνει κρατήσεις.
- **Συμμετέχων:** μπορεί να κάνει κρατήσεις σε ενεργές εκδηλώσεις, να βλέπει τις κρατήσεις του και να χρησιμοποιεί το σύστημα μηνυμάτων.
- **Διοργανωτής:** μπορεί να δημιουργεί, να επεξεργάζεται, να δημοσιεύει, να ακυρώνει και να διαχειρίζεται τις δικές του εκδηλώσεις.
- **Διαχειριστής:** μπορεί να εγκρίνει ή να απορρίπτει χρήστες και να εξάγει τα δεδομένα των εκδηλώσεων.

Η σύνδεση γίνεται με username και password. Μετά την επιτυχημένη σύνδεση, το backend εκδίδει JWT token. Το token αποστέλλεται από το frontend σε κάθε αίτημα προς προστατευμένα endpoints, και τα guards του NestJS (JwtAuthGuard, RolesGuard) ελέγχουν αν ο χρήστης έχει τον κατάλληλο ρόλο. Τα δημόσια endpoints σημειώνονται ρητά με τον decorator @Public().

Νέοι χρήστες που εγγράφονται δεν μπορούν να συνδεθούν αμέσως. Η κατάστασή τους είναι PENDING μέχρι να εγκριθούν από τον διαχειριστή.

5 Λειτουργίες Εφαρμογής

5.1 Καλωσόρισμα, Σύνδεση και Εγγραφή

Η πρώτη σελίδα της εφαρμογής δίνει πρόσβαση σε σύνδεση και εγγραφή. Αφού συνδεθεί, ο χρήστης βλέπει τους ρόλους του, συνδέσμους προς τις σελίδες που του αντιστοιχούν και προτεινόμενες εκδηλώσεις.

Η εγγραφή γίνεται σε τρία βήματα (λογαριασμός, στοιχεία, τοποθεσία). Ζητούνται τα στοιχεία της εκφώνησης: username, password με επιβεβαίωση, όνομα, επώνυμο, email, τηλέφωνο, ΑΦΜ, διεύθυνση και γεωγραφική θέση. Η θέση επιλέγεται πάνω σε χάρτη, με κλικ ή μετακίνηση του δείκτη. Κάθε βήμα ελέγχεται πριν τη μετάβαση στο επόμενο, ενώ ο έλεγχος ταύτισης των δύο κωδικών γίνεται σε επίπεδο φόρμας.

5.2 Διαχείριση Χρηστών

Ο διαχειριστής δημιουργείται κατά το seeding της βάσης. Από την κονσόλα διαχείρισης μπορεί να δει τις αιτήσεις σε αναμονή και να τις εγκρίνει ή να τις απορρίψει, να αναζητήσει χρήστες και να τους φιλτράρει ανά ρόλο (διοργανωτές, συμμετέχοντες), καθώς και να δει τα πλήρη στοιχεία κάθε χρήστη.

5.3 Διαχείριση Εκδηλώσεων

Οι διοργανωτές μπορούν να δημιουργούν νέες εκδηλώσεις με κατηγορίες, τύπους εισιτηρίων, χωρητικότητα, τοποθεσία και περιγραφή. Μια νέα εκδήλωση ξεκινά ως DRAFT και γίνεται ορατή μόνο όταν δημοσιευτεί. Οι βασικοί κανόνες ελέγχονται στο backend:

- μόνο εκδηλώσεις DRAFT με τουλάχιστον έναν τύπο εισιτηρίου μπορούν να δημοσιευτούν,
- μόνο δημοσιευμένες εκδηλώσεις μπορούν να ακυρωθούν,
- οι τύποι εισιτηρίων δεν αλλάζουν αφού υπάρχουν κρατήσεις,
- εκδήλωση με κρατήσεις δεν μπορεί να διαγραφεί.

Στην ίδια σελίδα ο διοργανωτής βλέπει τις κρατήσεις κάθε εκδήλωσής του.

5.4 Αναζήτηση και Προβολή Εκδηλώσεων

Οι χρήστες μπορούν να αναζητήσουν εκδηλώσεις με βάση κατηγορία, κείμενο, ημερομηνίες, τιμή και πόλη, με σελιδοποίηση των αποτελεσμάτων. Στο αίτημα αποστέλλονται μόνο τα φίλτρα που έχουν συμπληρωθεί. Η σελίδα λεπτομερειών παρουσιάζει τα πλήρη στοιχεία της εκδήλωσης και χάρτη OpenStreetMap με βάση τις αποθηκευμένες συντεταγμένες. Κάθε προβολή καταγράφεται και χρησιμοποιείται από τον αλγόριθμο συστάσεων.

5.5 Κρατήσεις

Η κράτηση απαιτεί επιβεβαίωση από τον χρήστη. Μετά την οριστική υποβολή, η κράτηση αποθηκεύεται και ενημερώνονται τα διαθέσιμα εισιτήρια μέσα σε ένα transaction. Ο έλεγχος γίνεται στο backend, ώστε να μην μπορεί να παρακαμφθεί από το frontend. Οι κρατήσεις είναι οριστικές και ο συμμετέχων τις βλέπει στη σελίδα «Οι κρατήσεις μου».

5.6 Μηνύματα

Υλοποιήθηκε σελίδα μηνυμάτων με εισερχόμενα, απεσταλμένα, κατάσταση ανάγνωσης, απάντηση και διαγραφή. Η γραμμή πλοήγησης εμφανίζει τον αριθμό των μη αναγνωσμένων μηνυμάτων,

ο οποίος ανανεώνεται κάθε 30 δευτερόλεπτα (polling), αφού το HTTP δεν επιτρέπει στον server να στείλει ειδοποίηση από μόνος του.

Κατά την ακύρωση εκδήλωσης, ο διοργανωτής γράφει ένα μήνυμα που αποστέλλεται σε όλους τους συμμετέχοντες με κράτηση, και στη συνέχεια η εκδήλωση ακυρώνεται («Cancel + notify»).

5.7 Εξαγωγή XML και JSON

Ο διαχειριστής μπορεί να εξάγει όλες τις εκδηλώσεις σε XML και JSON από την καρτέλα «Data Export» της κονσόλας διαχείρισης. Η έξοδος XML ακολουθεί τη δομή του DTD της εκφώνησης, με στοιχεία όπως Event, Category, TicketTypes, Bookings, Organizer και Media.

Τα endpoints εξαγωγής (/export/events.xml και /export/events.json) επιτρέπονται μόνο σε διαχειριστές. Γι' αυτό η λήψη γίνεται μέσω του HttpClient και όχι με απλό σύνδεσμο: έτσι ο AuthInterceptor προσθέτει το token και το αρχείο αποθηκεύεται τοπικά ως events.xml ή events.json.

6 Αλγόριθμος Συστάσεων

Ο αλγόριθμος συστάσεων υλοποιήθηκε από την αρχή, χωρίς έτοιμη βιβλιοθήκη machine learning. Χρησιμοποιήθηκε **Biased Matrix Factorization**, όπου κάθε χρήστης και κάθε εκδήλωση αναπαρίστανται με λανθάνοντα διανύσματα. Η πρόβλεψη ενδιαφέροντος του χρήστη u για την εκδήλωση i δίνεται από τη σχέση:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$$

όπου μ είναι ο συνολικός μέσος όρος, b_u και b_i οι αποκλίσεις (bias) χρήστη και εκδήλωσης, και $\mathbf{p}_u$, $\mathbf{q}_i$ τα λανθάνοντα διανύσματα.

Η εκπαίδευση γίνεται με stochastic gradient descent (learning rate 0.01, regularization 0.02, 50 epochs). Ως σήματα ενδιαφέροντος χρησιμοποιούνται:

- επιβεβαιωμένες κρατήσεις, με βάρος 1.0,
- προβολές εκδηλώσεων, με βάρος 0.3 ανά προβολή και μέγιστο 0.8,
- αρνητικά δείγματα (βάρος 0) για εκδηλώσεις με τις οποίες ο χρήστης δεν αλληλεπίδρασε.

Αν ένας χρήστης δεν έχει ιστορικό, η εφαρμογή επιστρέφει προτάσεις με βάση τη δημοφιλία των εκδηλώσεων (popularity fallback). Με αυτόν τον τρόπο εμφανίζονται προτάσεις ακόμη και όταν τα δεδομένα είναι περιορισμένα. Λόγω του μικρού μεγέθους των δεδομένων, το μοντέλο εκπαιδεύεται εκ νέου σε κάθε αίτημα ώστε το αποτέλεσμα να είναι πάντα ενημερωμένο. Για μεγάλα δεδομένα θα χρειαζόταν περιοδική εκπαίδευση.

Ο βασικός κώδικας βρίσκεται στα αρχεία:

- backend/src/recommendations/bmf.ts
- backend/src/recommendations/recommendations.service.ts

7 Εγκατάσταση και Εκτέλεση

Για την εκτέλεση χρειάζονται Node.js, npm, Docker και ένας σύγχρονος browser. Η πιο απλή εκκίνηση γίνεται από τον ριζικό φάκελο του project:

```
./run-all.sh
```

Το script εκτελεί τα βασικά βήματα αυτόματα και μπορεί να εκτελεστεί ξανά με ασφάλεια:

- δημιουργεί ή ξεκινά PostgreSQL container,

- δημιουργεί τοπικό `backend/.env` αν λείπει,
- δημιουργεί self-signed TLS certificates,
- εγκαθιστά τα dependencies,
- εκτελεί τα Prisma migrations,
- δημιουργεί demo χρήστες και demo εκδήλωση (seed),
- ξεκινά backend και frontend.

Με `Ctrl+C` σταματούν backend και frontend, ενώ το container της βάσης παραμένει ενεργό ώστε να μη χάνονται τα δεδομένα. Οι διευθύνσεις της εφαρμογής είναι:

- Backend: `https://localhost:3000`
- Frontend: `https://localhost:4200`

Οι demo λογαριασμοί είναι:

Username	Password	Ρόλος
admin	admin123	Διαχειριστής
organizer	organizer123	Διοργανωτής
user	user1234	Συμμετέχων

8 Έλεγχος και Δοκιμές

Για τον έλεγχο του backend χρησιμοποιήθηκαν unit tests. Κατά την τελική επαλήθευση πέρασαν και τα 14 test suites (36 tests):

```
cd backend
npm test
```

Επίσης έγινε build του frontend:

```
cd frontend
npm run build
```

Το build ολοκληρώθηκε επιτυχώς. Εμφάνιστηκαν μόνο δύο προειδοποιήσεις της Angular: ότι το αρχικό bundle ξεπερνά ελάχιστα το όριο μεγέθους (περίπου 906 kB έναντι 900 kB) και ότι η Leaflet είναι CommonJS dependency. Οι προειδοποιήσεις αυτές δεν εμποδίζουν τη λειτουργία της εφαρμογής.

Πέρα από τα αυτοματοποιημένα checks, έγινε χειροκίνητος έλεγχος των βασικών ροών: εγγραφή, έγκριση χρήστη, σύνδεση, δημιουργία εκδήλωσης, αναζήτηση, προβολή χάρτη, κράτηση, ανταλλαγή μηνυμάτων, εξαγωγή αρχείων και εμφάνιση συστάσεων.

9 Παραδοχές και Σχεδιαστικές Αποφάσεις

Κατά την υλοποίηση έγιναν οι παρακάτω παραδοχές:

- Τα TLS certificates είναι self-signed, επειδή η εφαρμογή εκτελείται τοπικά για σκοπούς ανάπτυξης και αξιολόγησης.
- Η βάση δεδομένων εκτελείται σε Docker container για να είναι εύκολη η εγκατάσταση.

- Το αρχείο `.env` δεν παραδίδεται, επειδή περιέχει τοπικές ρυθμίσεις. Το `run-all.sh` το δημιουργεί αυτόματα για τοπική εκτέλεση.
- Τα `node_modules`, τα build artifacts και τα παραγόμενα certificates δεν περιλαμβάνονται στο zip του πηγαίου κώδικα.
- Το dataset εισαγωγής δεν περιλαμβάνεται στο zip, εφόσον παρέχεται ξεχωριστά από το μάθημα.
- Οι κρατήσεις είναι οριστικές και δεν ακυρώνονται από τον συμμετέχοντα. Όταν ακυρώνεται μια εκδήλωση, οι υπάρχουσες κρατήσεις διατηρούνται και οι συμμετέχοντες ενημερώνονται με μήνυμα.

10 Δυσκολίες

Μία βασική δυσκολία ήταν η μετατροπή της περιγραφής XML/DTD σε σωστό σχεσιακό μοντέλο. Η λύση ήταν να χωριστούν οι επαναλαμβανόμενες πληροφορίες σε ξεχωριστούς πίνακες, όπως `TicketType`, `Booking`, `Category` και `Photo`.

Άλλη σημαντική δυσκολία ήταν η διατήρηση της σωστής διαθεσιμότητας εισιτηρίων. Για τον λόγο αυτό οι έλεγχοι γίνονται στο backend μέσα σε transactions και όχι μόνο στο frontend.

Η εξαγωγή αρχείων απαιτούσε επίσης προσοχή. Επειδή τα endpoints επιτρέπονται μόνο σε διαχειριστές, ένας απλός σύνδεσμος λήψης απέτυχε με 401, αφού ο browser δεν στέλνει το JWT token. Η λύση ήταν η λήψη μέσω `HttpClient`, ώστε το token να προστίθεται από τον interceptor.

Τέλος, ο αλγόριθμος συστάσεων απαιτούσε προσεκτικό χειρισμό των περιορισμένων δεδομένων. Γι' αυτό χρησιμοποιήθηκαν τόσο κρατήσεις όσο και προβολές εκδηλώσεων, καθώς και popularity fallback όταν δεν υπάρχει ιστορικό χρήστη.

11 Συμπέρασμα

Η εργασία ολοκληρώθηκε ως πλήρης εφαρμογή διαχείρισης εκδηλώσεων και ηλεκτρονικών κρατήσεων. Υλοποιήθηκαν οι βασικές απαιτήσεις της εκφώνησης, όπως ρόλοι χρηστών, έγκριση εγγράφων, CRUD εκδηλώσεων, έλεγχος χωρητικότητας, κρατήσεις, messaging, εξαγωγή XML/JSON και σύστημα συστάσεων.

Μέσα από την υλοποίηση έγινε πρακτική εφαρμογή τεχνολογιών web, σχεσιακής βάσης δεδομένων, REST API, JWT authentication και ενός απλού recommender system. Μελλοντικές βελτιώσεις θα μπορούσαν να περιλαμβάνουν αναλυτικότερο UI για στατιστικά διοργανωτών (π.χ. έσοδα ανά εκδήλωση), καλύτερη αναζήτηση με full-text indexes, ειδοποιήσεις σε πραγματικό χρόνο μέσω WebSockets και εκτενέστερη αξιολόγηση του αλγορίθμου συστάσεων με μεγαλύτερο dataset.